Seleção Inteligente - Branch and Bound

Repositório GitHub: [selecao_inteligente-branch_and_bound]

Alunos:

José Wagner Ramos da Silva | wagner.ramos@academico.ifpb.edu.br

Samuel Júnior Pereira Alcântara | junior.alcantara@academico.ifpb.edu.br

Paulo Oliveira Henrique Nunes | nunes.henrique@academico.ifpb.edu.br

Estrutura de Arquivos do Projeto

```
selecao_inteligente-branch_and_bound-main/
├── .gitignore
├── README.md
└── src/
    └── main/
        └── java/
            ├── Main.java
            ├── util/
            │   ├── Catalogo.java
            │   └── Validator.java
            ├── model/
            │   ├── No.java
            │   └── Projeto.java
            ├── service/
            │   ├── Resultado.java
            │   └── Selecao.java
            └── view/
                └── ConsoleUI.java
```

1. model/

1.1 No

Pseudocódigo

```
Classe No
    Atributos Privados:
        nivel: Inteiro
        beneficioAtual: Inteiro
        custoAtual: Real
        limiteSuperior: Real
        projetosAdotados: Lista de Texto

    // Construtor
    Construtor(nivel: Inteiro, beneficioAtual: Inteiro, custoAtual: Real,
projetosAdotados: Lista de Texto)
        este.nivel <- nivel
        este.beneficioAtual <- beneficioAtual
        este.custoAtual <- custoAtual
```

```
        este.limiteSuperior <- 0
        este.projetosAdotados <- nova Lista(projetosAdotados) // Cópias dos elementos
da lista
    FimConstrutor

    // Getters e Setters
```

Classe que representa o nó da árvore que o algoritmo irá percorrer. Com atributos para o nível, benefício atual do nó, custo, limite superior e projetos adotados.

2. model/

2.1 Projeto

Pseudocódigo

```
Classe Projeto
    Atributos Privados:
        codigo: Texto
        nome: Texto
        custo: Real
        beneficio: Inteiro
        obrigatorio: Booleano
        incompativeis: Lista de Texto

    // Construtor
    Construtor(codigo: Texto, nome: Texto, custo: Real, beneficio: Inteiro,
obrigatorio: Booleano)
        Validator.validarProjeto(codigo, nome, custo, beneficio, obrigatorio)
        este.codigo <- codigo
        este.nome <- nome
        este.custo <- custo
        este.beneficio <- beneficio
        este.obrigatorio <- obrigatorio
        este.incompativeis <- nova Lista()
    FimConstrutor

    // Método Estático
    MetodoEstatico tornarIncompativeis(a: Projeto, b: Projeto)
        Validator.validarObjeto(a, "Projeto")
        Validator.validarObjeto(b, "Projeto")
        a.incompativeis.adicionar(b.codigo)
        b.incompativeis.adicionar(a.codigo)
    FimMetodoEstatico
```

Classe que representa o projeto que estará presente em cada nó. Com atributos para o código, nome, custo, obrigatoriedade e incompatibilidade; Getters e Setters; Construtor e o método `tornarIncompativeis` que torna um par de projetos incompatíveis entre si.

3. service/

3.1 Resultado

PseudoCódigo

```
Classe Resultado
    Atributos Privados (Constantes):
        retornoTotal: InteiroLongo (ou Real)
        custoTotal: InteiroLongo (ou Real)
        projetosEscolhidos: Lista de Projeto

    // Construtor
    Construtor(retornoTotal: InteiroLongo, custoTotal: InteiroLongo,
projetosEscolhidos: Lista de Projeto)
        este.retornoTotal <- retornoTotal
        este.custoTotal <- custoTotal
        este.projetosEscolhidos <- projetosEscolhidos
    FimConstrutor

      //Getetrs
FimClasse
```

Classe para o resultado das operações, com os projetos que foram escolhidos. Possuindo atributos para o retorno e custo total desses projetos e um List dos projetos selecionados. Como também, construtor e métodos getters.

4. service/

4.1 Selecao

Classe utilizada para selecionar o projeto com base nas condições:

- A soma dos custos não pode ultrapassar o **orçamento informado**.
- Cada projeto pode ser escolhido no máximo uma vez; não são permitidas escolhas fracionadas.
- Se um cenário indicar projeto obrigatório, ele deverá integrar a seleção.

E restrição:

- **P4 e P7** não podem ser selecionados simultaneamente (incompatibilidade de infraestrutura).

Conta com 5 métodos:

1. No resolver(Projeto[] projetos, double orcamento)
2. No buscarMelhorSolucao(No raiz, Projeto[] projetos, double orcamento)
3. No ramificar(No atual, Projeto[] projetos, double orcamento, PriorityQueue<No> fila, No melhorNo)
4. double calcularLimiteSuperior(No no, Projeto[] projetos, double orcamento)
5. boolean temConflito(Projeto candidato, List<String> codigosAdotados)

1. resolver

Pseudocódigo

```
Funcao resolver(projetos: Vetor de Projeto, orcamento: Real): No
    Validator.validarLista(projetos)
    Validator.validarOrcamento(orcamento)

    opcionais <- nova Lista de Projeto()
    codigosObrigatorios <- nova Lista de Texto()
```

```
    custoObrigatorios <- 0.0
    beneficioObrigatorios <- 0

    Para cada projeto em projetos faça
        Se projeto.isObrigatorio() então
            Se temConflito(projeto, codigosObrigatorios) então
                lance Excecao("O projeto obrigatório " + projeto.getCodigo() + " é
incompatível com outro obrigatório")
            FimSe

            codigosObrigatorios.adicionar(projeto.getCodigo())
            custoObrigatorios <- custoObrigatorios + projeto.getCusto()
            beneficioObrigatorios <- beneficioObrigatorios + projeto.getBeneficio()
        Senão
            opcionais.adicionar(projeto)
        FimSe
    FimPara

    Se custoObrigatorios > orcamento então
        lance Excecao("Os projetos obrigatórios excedem o orçamento.")
    FimSe

    projetosOrdenados <- converterParaVetor(opcionais)

    // Ordena os projetos opcionais pela razão retorno/custo em ordem decrescente
    ordenarDecrescente(projetosOrdenados, razaoRetornoPorCusto)

    raiz <- novo No(nivel: -1, beneficioAtual: beneficioObrigatorios, custoAtual:
custoObrigatorios, projetosAdotados: codigosObrigatorios)

    raiz.setLimiteSuperior(calcularLimiteSuperior(raiz, projetosOrdenados,
orcamento))

    Retornar buscarMelhorSolucao(raiz, projetosOrdenados, orcamento)
FimFuncao
```

Esse método itera em nós “resolvendo” eles. Nesses nós estão presentes um array de projetos e o orçamento. O algoritmo verifica conflito entre projetos que são obrigatórios, caso não tenham, eles: `codigosObrigatorios.add(projeto.getCodigo());`, seu custo: `custoObrigatorios += projeto.getCusto();` e seu beneficio: `beneficioObrigatorios += projeto.getBeneficio();` são armazenados. Caso não seja obrigatório, `opcionais.add(projeto);` o projeto é armazenado em `opcionais`. Simultaneamente, é verificado se o custo dos projetos obrigatórios ultrapassa o orçamento (`custoObrigatorios > orcamento`) enquanto essa condição não for atendida os projetos são ordenados pela razão custo/beneficio. Após essas operações o limite superior do nó raiz é alterado e é retornado `buscarMelhorSolucao(raiz, projetosOrdenados, orcamento).`

2. buscarMelhorSolucao

Pseudocódigo

```
Funcao buscarMelhorSolucao(raiz: No, projetos: Vetor de Projeto, orcamento: Real): No
    // Cria uma fila de prioridade ordenada pelo limite superior em ordem decrescente
    fila <- nova FilaDePrioridade ordenada por No::getLimiteSuperior em ordem
```

```
decrescente

    fila.adicionar(raiz)

    melhorNo <- novo No(raiz.getNivel(), raiz.getBeneficioAtual(),
raiz.getCustoAtual(), raiz.getProjetosAdotados())

    Enquanto fila não estiver vazia faça
        atual <- fila.removerProximo() // Remove o nó com maior limite superior

        // Poda: se o limite superior do nó atual for menor ou igual ao benefício da
melhor solução, encerra a busca
        Se atual.getLimiteSuperior() <= melhorNo.getBeneficioAtual() então
            pare
        FimSe

        melhorNo <- ramificar(atual, projetos, orcamento, fila, melhorNo)
    FimEnquanto

    Retornar melhorNo
FimFuncao
```

Esse metodo busca a melhor solução (melhor nó) para o problema passando pelos nós da árvore, analisando orçamento, custo e beneficio. Primeiramente, é criada uma fila de prioridade baseada no limite superior de cada nó e assumimos que a melhor escolha atual é o nó raiz: `melhorNo <- novo No(raiz.getNivel(), raiz.getBeneficioAtual(), raiz.getCustoAtual(), raiz.getProjetosAdotados())`. Em seguida, enquanto a fila não estiver vazia, o algoritmo remove o nó do topo e verifica a condição de poda: Se `atual.getLimiteSuperior() <= melhorNo.getBeneficioAtual()`,o algoritmo ramifica o nó atual para avaliar suas ramificações (filhos), atualizando o `melhorNo` se necessário, e repete o processo até esvaziar a fila ou realizar todas as podas, retornando ao final o melhor nó encontrado. Caso contrário, o loop é interrompido (pare).

3. ramificar

Pseudocódigo

```
Funcao ramificar(atual: No, projetos: Vetor de Projeto, orcamento: Real, fila:
FilaDePrioridade, melhorNo: No): No
    proximoNivel <- atual.getNivel() + 1

    Se proximoNivel >= projetos.tamanho então
        Retornar melhorNo
    FimSe

    projeto <- projetos[proximoNivel]

    cabeNoOrcamento <- (atual.getCustoAtual() + projeto.getCusto() <= orcamento)
    semConflito <- NAO temConflito(projeto, atual.getProjetosAdotados())

    Se cabeNoOrcamento E semConflito então
        adotados <- nova Lista de Texto(atual.getProjetosAdotados())
        adotados.adicionar(projeto.getCodigo())

        comProjeto <- novo No(
```

```
            proximoNivel,
            atual.getBeneficioAtual() + projeto.getBeneficio(),
            atual.getCustoAtual() + projeto.getCusto(),
            adotados
        )

        comProjeto.setLimiteSuperior(calcularLimiteSuperior(comProjeto, projetos,
orcamento))

        Se comProjeto.getBeneficioAtual() > melhorNo.getBeneficioAtual() então
            melhorNo <- comProjeto
        FimSe

        Se comProjeto.getLimiteSuperior() > melhorNo.getBeneficioAtual() então
            fila.adicionar(comProjeto)
        FimSe
    FimSe

    semProjeto <- novo No(
        proximoNivel,
        atual.getBeneficioAtual(),
        atual.getCustoAtual(),
        atual.getProjetosAdotados()
    )

    semProjeto.setLimiteSuperior(calcularLimiteSuperior(semProjeto, projetos,
orcamento))

    Se semProjeto.getLimiteSuperior() > melhorNo.getBeneficioAtual() então
        fila.adicionar(semProjeto)
    FimSe

    Retornar melhorNo
FimFuncao
```

Esse método analisa se o nó atual(nó ramificado) + o proximo nó, cumprem os requisitos para serem adicionados aos selecionados. No início, é verificado se o proximo nivel que será analisado, é maior que a quantidade de projetos (o que significa que que estamos nas folhas da árvore), caso aconteça: `Retornar melhorNo`. Em seguida, é analisado se o proximo cabe no orçamento e não tem nenhum conflito com os projetos já selecionados, caso: `cabeNoOrcamento E semConflito == true`, ele é adicionado aos projetos adotados e os atributos do seu projeto são adicionados a um nó. Com isso, o limite superior é alterado, caso `comProjeto.getBeneficioAtual() > melhorNo.getBeneficioAtual()`, então, `melhorNo <- comProjeto` e `comProjeto.getLimiteSuperior() > melhorNo.getBeneficioAtual()`, então, `fila.adicionar(comProjeto)`. Se o próximo não couber no orçamento ou tiver conflito, é apenas criado um nó sem projeto, alterado o limite superior e se `semProjeto.getLimiteSuperior() > melhorNo.getBeneficioAtual()` , então, `fila.adicionar(semProjeto)`. Por fim, o melhor nó, com algum projeto válido foi encontrado, então `Retornar melhorNo`.

4. calcularLimiteSuperior

```
 Funcao calcularLimiteSuperior(no: No, projetos: Vetor de Projeto, orcamento: Real):
Real
```

```
    Se no.getCustoAtual() > orcamento então
        Retornar 0
    FimSe

    limite <- no.getBeneficioAtual()
    custoTotal <- no.getCustoAtual()
    j <- no.getNivel() + 1

    // Adiciona os projetos inteiros enquanto couberem no orçamento restante
    Enquanto j < projetos.tamanho E custoTotal + projetos[j].getCusto() <= orcamento
faça
        custoTotal <- custoTotal + projetos[j].getCusto()
        limite <- limite + projetos[j].getBeneficio()
        j <- j + 1
    FimEnquanto

    // Se ainda restou orçamento, pega a fração (razão) do próximo projeto (abordagem
gulosa/fracionária)
    Se j < projetos.tamanho então
        razao <- (Real) projetos[j].getBeneficio() / projetos[j].getCusto()
        limite <- limite + (orcamento - custoTotal) * razao
    FimSe

    Retornar limite
FimFuncao
```

Esse método é utilizado para calcular o limite superior da árvore de projetos conforme o algoritmo escolhe e descarta nós. Inicialmente, é verificado se o custo do nó atual ultrapassa o limite, se for verdadeiro: `Retornar 0`. A partir do próximo nivel, enquando os nós a serem analisados não acabarem e o custo total + custo do proximo projeto não ultrapassarem o orçamento, o custo total é atribuido com o custo do proximo e o limite com o benefício do próximo. Se ainda restar orçamento e houver projetos restantes que não couberem inteiros, é feita a fração proporcional (razão benefício/custo) desse próximo projeto para completar o limite superior, que é então retornado.

5. temConflito

```
Funcao temConflito(candidato: Projeto, codigosAdotados: Lista de Texto): Booleano
    Para cada codigo em codigosAdotados faça
        Se candidato.getIncompativeis().contem(codigo) então
            Retornar verdadeiro
        FimSe
    FimPara

    Retornar falso
FimFuncao
    private static boolean temConflito(
            Projeto candidato, List<String> codigosAdotados) {

        for (String codigo : codigosAdotados) {
            if (candidato.getIncompativeis().contains(codigo)) {
                return true;
            }
        }
```

```
        return false;
    }
```
Esse método analisa se há conflito entre um projeto candidato e os projetos já adotados

5. util/

5.1 Validator

Pseudocódigo

```
Classe Validator
    // Classe utilitária: todos os métodos são estáticos

    MetodoEstatico validarObjeto(objeto: Objeto, nome: Texto)
        Se objeto = nulo então
            lance ArgumentoInvalido(nome + " não pode ser nulo")
        FimSe
    FimMetodoEstatico

    MetodoEstatico validarString(texto: Texto, nome: Texto)
        Se texto = nulo OU texto está vazio ou só com espaços então
            lance ArgumentoInvalido(nome + " não pode ser nulo, vazio ou conter
somente espaços")
        FimSe
    FimMetodoEstatico

    MetodoEstatico validarCusto(custo: Real)
        Se custo <= 0 então
            lance ArgumentoInvalido("Custo deve ser maior que zero.")
        FimSe
    FimMetodoEstatico

    MetodoEstatico validarBeneficio(beneficio: Inteiro)
        Se beneficio < 0 então
            lance ArgumentoInvalido("Beneficio não pode ser menor que 0.")
        FimSe
    FimMetodoEstatico

    MetodoEstatico validarProjeto(codigo: Texto, nome: Texto, custo: Real, beneficio:
Inteiro, obrigatorio: Booleano)
        validarString(codigo, "Codigo")
        validarString(nome, "Nome")
        validarCusto(custo)
        validarBeneficio(beneficio)
    FimMetodoEstatico

    MetodoEstatico validarLista(lista: Vetor de Objeto)
        Se lista = nulo então
            lance ArgumentoInvalido("Lista não pode ser nula.")
        FimSe

        Para cada item em lista faça
            validarObjeto(item, "item da lista")
        FimPara
```

```
    FimMetodoEstatico

    MetodoEstatico validarOrcamento(orcamento: Real)
        Se orcamento <= 0 então
            lance ArgumentoInvalido("Orçamento tem que ser positivo.")
        FimSe
    FimMetodoEstatico
FimClasse
```

Classe utilitária que centraliza as validações do sistema, evitando repetir os mesmos `if` em várias classes. Todos os métodos são estáticos e lançam uma `IllegalArgumentException` com uma mensagem descritiva quando o dado é inválido. Conta com 7 métodos:

- `validarObjeto(Object objeto, String nome)`: garante que o objeto não é nulo.
- `validarString(String texto, String nome)`: garante que o texto não é nulo, vazio ou composto só por espaços.
- `validarCusto(double custo)`: o custo deve ser maior que zero.
- `validarBeneficio(int beneficio)`: o benefício não pode ser negativo.
- `validarProjeto(...)`: agrupa as validações de código, nome, custo e benefício. É chamado no construtor de `Projeto`.
- `validarLista(Object[] lista)`: a lista não pode ser nula e nenhum item dela pode ser nulo. É usado em `Selecao.resolver`.
- `validarOrcamento(double orcamento)`: o orçamento deve ser positivo. Também é usado em `Selecao.resolver`.

5.2 Catalogo

Pseudocódigo

```
Classe Catalogo
    Atributos Privados (Constantes):
        projetos: Mapa de Texto para Projeto // mantém a ordem de inserção

    // Construtor padrão: inicia o mapa vazio

    Metodo adicionar(projeto: Projeto)
        Validator.validarObjeto(projeto, "Projeto")

        Se projetos já contém projeto.getCodigo() então
            lance Excecao("Código duplicado: " + projeto.getCodigo())
        FimSe

        projetos.inserir(projeto.getCodigo(), projeto)
    FimMetodo

    Funcao buscar(codigo: Texto): Projeto
        Validator.validarString(codigo, "Codigo do projeto")

        projeto <- projetos.obter(codigo)

        Se projeto = nulo então
            lance Excecao("Projeto não encontrado: " + codigo)
        FimSe
```

```
        Retornar projeto
    FimFuncao

    Metodo tornarIncompativeis(codigoA: Texto, codigoB: Texto)
        Projeto.tornarIncompativeis(buscar(codigoA), buscar(codigoB))
    FimMetodo

    Metodo removerIncompativeis(codigoA: Texto, codigoB: Texto)
        Projeto.removerIncompatibilidade(buscar(codigoA), buscar(codigoB))
    FimMetodo

    Metodo remover(codigo: Texto)
        Validator.validarString(codigo, "Código do projeto")

        Se projetos.remover(codigo) = nulo então
            lance Excecao("Projeto não encontrado: " + codigo)
        FimSe

        // Remove o código dos demais projetos para não sobrar incompatibilidade
"fantasma"
        Para cada p em projetos.valores() faça
            p.getIncompativeis().remover(codigo)
        FimPara
    FimMetodo

    Metodo modificar(codigo: Texto, custo: Real, beneficio: Inteiro)
        modificado <- buscar(codigo)
        Validator.validarCusto(custo)
        Validator.validarBeneficio(beneficio)

        modificado.setCusto(custo)
        modificado.setBeneficio(beneficio)
    FimMetodo

    Funcao getProjetosMap(): Mapa de Texto para Projeto
        Retornar projetos
    FimFuncao

    Funcao comoArray(): Vetor de Projeto
        Retornar converterParaVetor(projetos.valores())
    FimFuncao
FimClasse
```

Classe que funciona como o cadastro (repositório em memória) dos projetos. Os projetos são guardados em um `LinkedHashMap`, usando o código como chave, o que impede códigos repetidos e mantém a ordem em que foram cadastrados. Conta com 7 métodos:

1. `void adicionar(Projeto projeto)`
2. `Projeto buscar(String codigo)`
3. `void tornarIncompativeis(String codigoA, String codigoB)`
4. `void removerIncompativeis(String codigoA, String codigoB)`
5. `void remover(String codigo)`
6. `void modificar(String codigo, double custo, int beneficio)`

7. `Projeto[] comoArray()` (além do getter `getProjetosMap()`)

Os métodos funcionam da seguinte forma: `adicionar` valida o projeto e usa `putIfAbsent`, lançando exceção se o código já existir. `buscar` valida o código e retorna o projeto, ou lança exceção caso ele não exista; ele é reaproveitado pelos outros métodos. `tornarIncompativeis` e `removerIncompativeis` buscam os dois projetos pelo código e delegam para os métodos estáticos de `Projeto`. `remover` apaga o projeto do mapa e, em seguida, percorre os demais projetos retirando esse código de suas listas de incompatíveis. `modificar` altera apenas custo e benefício, validando os novos valores. Por fim, `comoArray` converte os projetos para um vetor, que é o formato esperado por `Selecao.resolver`.

6. view/

6.1 ConsoleUI

Pseudocódigo

```
Classe ConsoleUI
    Atributos Privados:
        catalogo: Catalogo (constante)
        orcamentoPadrao: Real <- 50000.0
        scanner: Leitor de Entrada (constante)

    // Construtor
    Construtor(catalogo: Catalogo)
        este.catalogo <- catalogo
    FimConstrutor

    Metodo iniciar()
        opcao <- -1

        Repita
            exibirMenu()

            Tente
                opcao <- converterParaInteiro(scanner.lerLinha())
            Capture FormatoNumericoInvalido
                escreva("Opção inválida! Digite um número.")
                continue
            FimTente

            Escolha opcao
                caso 1: listarProjetos()
                caso 2: adicionarProjeto()
                caso 3: removerProjeto()
                caso 4: modificarProjeto()
                caso 5: mudarObrigatorio()
                caso 6: adicionarIncompatibilidade()
                caso 7: removerIncompatibilidade()
                caso 8: alterarOrcamento()
                caso 9: executarSelecao()
                caso 0: escreva("Saindo do sistema... Até logo!")
                outro caso: escreva("Opção desconhecida.")
            FimEscolha
```

```
        Enquanto opcao != 0
    FimMetodo

    Metodo listarProjetos()
        Se catalogo.getProjetosMap() está vazio então
            escreva("(Nenhum projeto cadastrado)")
            Retornar
        FimSe

        Para cada p em catalogo.getProjetosMap().valores() faça
            escreva(p.getCodigo(), p.getNome(), p.getCusto(), p.getBeneficio(),
p.isObrigatorio())

            Se p.getIncompativeis() não está vazia então
                escreva("Incompatíveis com: " + juntar(p.getIncompativeis(), ", "))
            FimSe
        FimPara
    FimMetodo

    Metodo adicionarProjeto()
        Tente
            codigo <- minusculas(scanner.lerLinha())
            nome <- scanner.lerLinha()
            custo <- converterParaReal(scanner.lerLinha())
            beneficio <- converterParaInteiro(scanner.lerLinha())
            obrigatorio <- (resposta "s/n" começa com "s")

            catalogo.adicionar(novo Projeto(codigo, nome, custo, beneficio,
obrigatorio))
            escreva("Projeto adicionado com sucesso!")
        Capture Excecao e
            escreva("Erro ao adicionar projeto: " + e.mensagem)
        FimTente
    FimMetodo

    Metodo removerProjeto()
        Tente
            codigo <- minusculas(scanner.lerLinha())
            catalogo.remover(codigo)
            escreva("Projeto removido com sucesso!")
        Capture Excecao e
            escreva("Erro ao remover projeto: " + e.mensagem)
        FimTente
    FimMetodo

    Metodo modificarProjeto()
        Tente
            codigo <- minusculas(scanner.lerLinha())
            projeto <- catalogo.buscar(codigo)

            escreva("Custo atual: " + projeto.getCusto())
            custo <- converterParaReal(trocar(scanner.lerLinha(), ",", "."))

            escreva("Beneficio atual: " + projeto.getBeneficio())
            beneficio <- converterParaInteiro(scanner.lerLinha())
```

```
        catalogo.modificar(codigo, custo, beneficio)
        escreva("Projeto modificado com sucesso!")
    Capture Excecao e
        escreva("Erro ao modificar projeto: " + e.mensagem)
    FimTente
FimMetodo

Metodo mudarObrigatorio()
    Tente
        codigo <- minusculas(scanner.lerLinha())
        p <- catalogo.buscar(codigo)
        p.setObrigatorio(NAO p.isObrigatorio())
        escreva("Status do projeto alterado")
    Capture Excecao e
        escreva("Erro: " + e.mensagem)
    FimTente
FimMetodo

Metodo adicionarIncompatibilidade()
    Tente
        c1 <- minusculas(scanner.lerLinha())
        c2 <- minusculas(scanner.lerLinha())
        catalogo.tornarIncompativeis(c1, c2)
        escreva("Projetos agora são incompatíveis!")
    Capture Excecao e
        escreva("Erro: " + e.mensagem)
    FimTente
FimMetodo

Metodo removerIncompatibilidade()
    Tente
        c1 <- minusculas(scanner.lerLinha())
        c2 <- minusculas(scanner.lerLinha())
        catalogo.removerIncompativeis(c1, c2)
        escreva("Incompatibilidade removida!")
    Capture Excecao e
        escreva("Erro: " + e.mensagem)
    FimTente
FimMetodo

Metodo alterarOrcamento()
    Tente
        novoOrcamento <- converterParaReal(scanner.lerLinha())

        Se novoOrcamento <= 0 então
            lance Excecao("O orçamento deve ser maior que zero.")
        FimSe

        orcamentoPadrao <- novoOrcamento
        escreva("Orçamento alterado com sucesso")
    Capture Excecao e
        escreva("Erro ao alterar orçamento: " + e.mensagem)
    FimTente
FimMetodo
```

```
    Metodo executarSelecao()
        Tente
            melhor <- Selecao.resolver(catalogo.comoArray(), orcamentoPadrao)

            escreva("Orçamento limite: " + orcamentoPadrao)
            escreva("Benefício máximo: " + melhor.getBeneficioAtual())
            escreva("Custo total: " + melhor.getCustoAtual())

            Se melhor.getProjetosAdotados() está vazia então
                escreva("(Nenhum projeto selecionado)")
            Senão
                Para cada codigo em melhor.getProjetosAdotados() faça
                    Tente
                        p <- catalogo.buscar(codigo)
                        escreva(p.getCodigo(), p.getNome(), p.getCusto(),
p.getBeneficio())
                    Capture Excecao
                        escreva(codigo)
                    FimTente
                FimPara
            FimSe
        Capture Excecao e
            escreva("Erro na seleção: " + e.mensagem)
        FimTente
    FimMetodo
FimClasse
```

Classe responsável pela interface de console (menu interativo) do sistema. Ela recebe o `Catalogo` pelo construtor, guarda o orçamento atual (começa em R$ 50.000,00) e usa um `Scanner` para ler o que o usuário digita. Conta com os métodos:

1. `void iniciar()`
2. `void listarProjetos()`
3. `void adicionarProjeto()`
4. `void removerProjeto()`
5. `void modificarProjeto()`
6. `void mudarObrigatorio()`
7. `void adicionarIncompatibilidade()`
8. `void removerIncompatibilidade()`
9. `void alterarOrcamento()`
10. `void executarSelecao()`

O método `iniciar` exibe o menu em loop, lê a opção digitada (tratando entradas que não sejam números) e chama o método correspondente, até o usuário escolher `0` para sair. `listarProjetos` mostra código, nome, custo, benefício, se é obrigatório e os projetos incompatíveis de cada item. `adicionarProjeto`, `removerProjeto` e `modificarProjeto` leem os dados e delegam ao `Catalogo`; os códigos digitados são convertidos para minúsculas. `mudarObrigatorio` inverte o status de obrigatoriedade de um projeto. `adicionarIncompatibilidade` e `removerIncompatibilidade` pedem dois códigos e chamam o `Catalogo`. `alterarOrcamento` troca o orçamento usado na seleção, rejeitando valores menores ou iguais a zero. Por fim, `executarSelecao` chama

`Selecao.resolver(catalogo.comoArray(), orcamentoPadrao)` e imprime o benefício máximo, o custo total e os projetos adotados do melhor nó encontrado. Todos os métodos que dependem de entrada do usuário capturam exceções e mostram a mensagem de erro, para que o programa não seja encerrado por um dado inválido.

7. Main

Pseudocódigo

```
Programa Principal
    Metodo principal()
        catalogo <- novo Catalogo()

        catalogo.adicionar(novo Projeto("P1", "Monetização do Laboratórios", 18000,
90, falso))
        catalogo.adicionar(novo Projeto("P2", "Plataforma de Ensino", 12000, 65,
falso))
        catalogo.adicionar(novo Projeto("P3", "Rede sem fio institucional", 10000,
55, falso))
        catalogo.adicionar(novo Projeto("P4", "Servidores de Pesquisa", 22000, 110,
falso))
        catalogo.adicionar(novo Projeto("P5", "Sistema de Segurança", 8000, 40,
falso))
        catalogo.adicionar(novo Projeto("P6", "Capacitação Docente", 6000, 35,
falso))
        catalogo.adicionar(novo Projeto("P7", "Laboratório de IA", 20000, 105,
falso))
        catalogo.adicionar(novo Projeto("P8", "Sistema de Gestão Acadêmica", 14000,
75, falso))

        ui <- novo ConsoleUI(catalogo)
        ui.iniciar()
    FimMetodo
FimPrograma
```

Classe principal, ponto de entrada do programa. Ela cria um `Catalogo` e o preenche com os 8 projetos do cenário de exemplo (P1 a P8), todos inicialmente opcionais. Depois, cria a `ConsoleUI` passando o catálogo e chama `iniciar()`, que abre o menu interativo. A partir daí, o usuário pode alterar os projetos, definir obrigatoriedades e incompatibilidades (como a de P4 e P7), ajustar o orçamento e executar a seleção por Branch and Bound.